GuardCode: Suppressing Undetected Frame Errors
with a Soft-Syndrome Guard on CRC-Protected Neural FEC Links

Project Report
BCSE308L - Computer Networks
BCSE308P - Computer Networks Lab

Submitted by
Naresh Nireenjan (24BCE2353)

Under the kind guidance of
Dr. Sugumar R

School of Computer Science and Engineering
Vellore Institute of Technology, Vellore
October, 2026

Abstract

Every frame on a data link carries a cyclic redundancy check (CRC). The receiver decodes the frame, recomputes the CRC and either delivers the frame or asks for a retransmission. The weak spot of this design is the **undetected error**: a frame that is decoded wrongly but still happens to satisfy the CRC, so corrupted data is delivered silently. This project builds and evaluates **GuardCode**, a receiver that closes much of that gap without adding a single bit to the frame and without changing the CRC polynomial.

The key component, the **Soft-Syndrome Guard (SSG)**, uses the decoder's soft output and the exact algebra of the standard CRC to compute, for every frame that passes the CRC, the probability that it is still wrong. Frames whose residual risk is too high are retransmitted. A learned neural channel code (32-bit payload, CRC-8, 70 channel symbols) is trained in PyTorch and gains about 1 dB over Hamming(7,4) at the same rate. The system is evaluated by Monte-Carlo simulation on AWGN and Gilbert-Elliott burst channels with 3,00,000 frames per operating point. At an identical retransmission budget, SSG removes 48 % (AWGN) and 50 % (burst) of the undetected errors left by the classic confidence-threshold test, and 80 % / 49 % of those of a plain Hamming receiver. A channel-adaptive controller, a live web dashboard and a SQLite database that logs every simulated frame complete the system. A provisional patent specification is drafted from the work.

Keywords: CRC, undetected error probability, ARQ, forward error correction, neural channel codes, Hamming code, burst errors, data link layer.

1. Introduction

1.1 Background

Module 3 of the course covers error detection and correction (parity, checksum, CRC, Hamming code) and flow and error control (Stop-and-Wait, Go-Back-N and Selective-Repeat ARQ). In a real link these mechanisms work together. A forward error correction (FEC) code repairs most channel errors. The CRC then decides whether the repaired frame can be trusted. ARQ retransmits the frames the CRC rejects.

A CRC with r check bits misses roughly one in 2^r of the error patterns that reach it. For short CRCs used in low-latency, IoT and control traffic (CRC-8 to CRC-16), this residual is not negligible: one undetected error per few thousand frame errors is enough to corrupt a sensor stream or a control command. The usual remedies have real costs:

- a longer CRC costs payload on every frame;
- a redesigned polynomial breaks compatibility with standards;
- simple confidence thresholds reject many good frames.

1.2 Problem statement

Can a receiver lower the undetected-error rate of a CRC-protected link, at a fixed retransmission budget, without extra redundancy and without changing the CRC or the frame format?

1.3 Objectives

1. Build a link-level simulator: CRC-8 framing, Hamming(7,4) and learned neural FEC, AWGN, burst and fading channels, and ARQ accounting.

2. Design a receiver test that uses the CRC's algebraic structure and the decoder's soft output to estimate the residual risk of each frame (the Soft-Syndrome Guard).
3. Compare it fairly with plain CRC acceptance and with the classic minimum-|LLR| threshold test, at equal retransmission cost.
4. Add a channel-adaptive profile controller, a live monitoring dashboard and a large frame database.
5. Assess patentability and draft a provisional patent specification.

1.4 Mapping to the syllabus

Course topic	Where it appears in this project
Module 3 - CRC (polynomial division)	guardcode/crc.py: bit-serial CRC register, generator matrix, parity-check matrix
Module 3 - Hamming code	guardcode/hamming.py: Hamming(7,4), soft ML decoding, block interleaver
Module 3 - Burst errors, interleaving	Gilbert-Elliott burst channel; interleaved Hamming and neural profiles
Module 3 - ARQ (Selective Repeat)	Goodput model in linksim.py; NAK on CRC or guard rejection
Module 1 - Transmission impairments	AWGN, burst noise and Rayleigh fading channel models
Module 6 - QoS, reliability	Residual (undetected) error rate as a QoS metric; risk ledger
Lab - socket/web programming	Flask web server with Server-Sent Events, REST API, SQL explorer

2. Literature Survey and Prior Art

Work	Idea	Limitation addressed by GuardCode
CRC polynomial design for a given inner code (Lou, Wesel et al., UCLA)	Choose g(x) to minimise undetected errors for a convolutional code	Requires changing the standard polynomial; GuardCode keeps g(x)
CRC-aided list decoding (polar CA-SCL; US 2023/0198672 A1, list autoencoders)	CRC selects among decoder candidates; CRC treated as data bits during training	Decoder-specific; does not estimate residual risk of the accepted frame
Turbo Autoencoder, DeepJSCC, hierarchical deep channel coding (US 12155482 B2)	Neural channel codes trained end-to-end	Use parity/LLR checks only as runtime flags
Forney erasure decoding; ROVA (Raghavan and Baum); threshold tests for polar UER	Reject when decoder confidence is low	Confidence is blind to which errors the CRC would catch
Gallager's parity-check probability (LDPC)	Probability that a parity check is satisfied from bit probabilities	Used inside iterative decoders, not as a CRC acceptance test

Table 1 - Related work

A search of Google Patents, Justia and arXiv found no document that computes the CRC-conditioned posterior probability of an accepted frame being wrong through the dual-code expansion of a standard CRC and uses it as an ARQ acceptance gate. The closest patent, US 2023/0198672 A1, explicitly treats CRC bits as information bits during training and uses the CRC only to check list candidates. Section 7 discusses what this means for patentability.

3. Proposed System

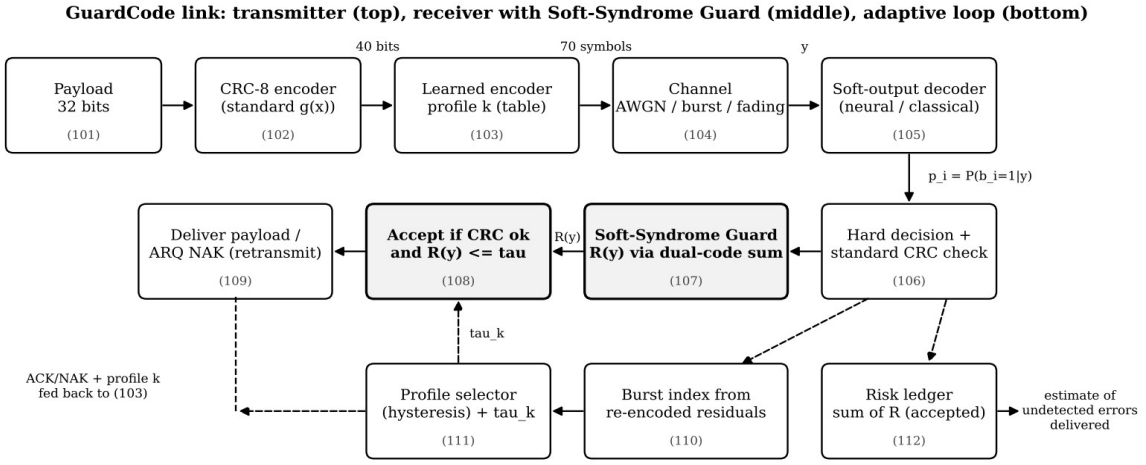


Figure 1 - GuardCode system architecture

3.1 Frame format and CRC as a linear code

A frame carries k = 32 payload bits and r = 8 CRC bits with g(x) = x^8 + x^2 + x + 1. Since the CRC (zero initial value) is linear, crc(m) = m G (mod 2), where row i of G is the CRC of the i-th unit vector, which is exactly the long division done in the lab. Every valid 40-bit frame b satisfies H b = 0 with H = [G^T | I_8]. Exhaustive search shows the minimum distance of this frame code is 4: every pattern of up to 3 bit errors is always detected.

3.2 Soft-Syndrome Guard

The decoder outputs a posterior over each **group** of frame bits: the 256 values of a byte for the neural decoder, the 16 messages of a 4-bit Hamming block, or one bit for uncoded BPSK. The hard decision x takes the most likely value of every group. Knowing that the transmitted frame must be a CRC codeword, Bayes' rule gives the probability that the decision is right:

$$\mathrm{P}(\mathrm{b} = \mathrm{x} \mid \mathrm{y}, \mathrm{Hb} = 0) = \prod\nolimits_{\mathrm{c}} \mathrm{P_c}(\mathrm{x_c}) \;/\; \mathrm{P}(\mathrm{Hb} = 0 \mid \mathrm{y})$$

The denominator is the probability that a frame drawn from the decoder's posterior satisfies the CRC. It is computed exactly by summing over the 2^r = 256 words u of the dual code. Each group contributes one value of the Walsh-Hadamard transform (WHT) of its posterior:

$$\mathrm{P}(\mathrm{Hb} = 0 \mid \mathrm{y}) = 2\hat{}(-8) \cdot \Sigma_\mathrm{u} \prod\nolimits_{\mathrm{c}} \mathrm{F_c}(\mathrm{s_c}(\mathrm{u})), \quad \mathrm{F_c}(\mathrm{s}) = \Sigma_\mathrm{v}\ \mathrm{P_c}(\mathrm{v})\ (-1)\hat{}\mathrm{popcount}(\mathrm{s\ AND\ v})$$

Here s_c(u) is the part of the row u^T H that falls inside group c. The residual risk is R(y) = 1 − P(b = x | y, Hb = 0). A frame is delivered only if the CRC passes **and** R(y) ≤ τ; otherwise a NAK triggers retransmission. The cost per frame is one 256-point WHT per byte plus a 256 x 5 gather-and-product, which is negligible next to the decoder. Two numerical checks confirmed the formula:

- against a 400,000-trial Monte-Carlo estimate: 0.003671 analytical vs 0.003663 simulated;
- for correlated byte posteriors around a CRC-valid decision: 0.3236 analytical vs 0.3246 simulated (1,000,000 trials).

Why groups matter. A simpler variant treats the 40 bits as independent. That ignores the fact that bits inside the same byte or Hamming block fail together, so its risk estimate is badly miscalibrated. The group/WHT form keeps that correlation exactly; Section 6.3 compares the two.

Why SSG beats a confidence threshold. A minimum-|LLR| test rejects a frame if any single bit is uncertain, even when that uncertainty is confined to patterns the CRC would certainly catch. SSG only counts error patterns that the CRC cannot see, so it spends the retransmission budget on the frames that are actually dangerous.

3.3 Learned neural channel code

- **Encoder:** the 40-bit frame is cut into 5 bytes. A trainable table maps each byte (256 values) to 14 real symbols, giving 70 symbols (rate 32/70, the same as Hamming(7,4)). The frame is normalised to unit energy per symbol. The burst profile adds a fixed interleaver.
- **Decoder:** a 3-layer MLP (512 ELU units per layer) sees all 70 received symbols and outputs 256-way log-posteriors for each byte. Bit posteriors are their marginals, which is what SSG needs.
- **Training:** 5,000 Adam steps, batch 1,024, Eb/N0 uniform in 1-6 dB, through a differentiable channel (PyTorch). Weights are exported to .npz so the simulator and dashboard run on numpy only.

3.4 Channel-adaptive profile controller

Two profiles are trained, one for AWGN and one for Gilbert-Elliott bursts. After decoding, the receiver re-encodes the decision and measures residual energy in windows of 7 symbols. The burst index is log10(max/median) of those window energies. An EWMA (factor 0.02) with hysteresis (switch up at 0.44, down at 0.40) selects the profile, which is fed back in the ACK. Each profile has its own calibrated τ.

3.5 Risk ledger

Summing R(y) over frames gives the receiver its own estimate of how many undetected errors it has delivered, without knowing the transmitted data. Section 6.4 checks how far this estimate can be trusted.

4. Implementation

Component	File	Technology
CRC, parity-check and dual-code matrices	guardcode/crc.py	Python, numpy
Channels (AWGN, Gilbert-Elliott, Rayleigh)	guardcode/channels.py	numpy + differentiable torch version
Hamming(7,4) soft decoder, interleaver	guardcode/hamming.py	numpy
Neural code: training / inference	guardcode/model.py, train.py	PyTorch / numpy
Soft-Syndrome Guard	guardcode/guard.py	numpy (log-domain matrix products)
Adaptive controller	guardcode/adaptive.py	numpy
Link simulator, benchmark, plots	guardcode/linksim.py, evaluate.py	numpy, matplotlib
Database layer	guardcode/db.py	SQLite (WAL mode)
Live dashboard and SQL explorer	dashboard/app.py, engine.py, templates	Flask, Server-Sent Events, Chart.js
Bulk database population, analyses	scripts/populate_db.py, analysis.py	Python

Table 2 - Software modules

4.1 Database design

Every transmitted frame, from bulk runs and from live operation, is stored as one row. The database runs in WAL (write-ahead logging) mode, so the live simulator writes about 1,200 rows per second while the dashboard reads. Setup populates 1,000,000 frames. A retention policy keeps the newest 10 million rows during live use.

Table	Key columns	Purpose
frames	run_id, ts, seq, scheme, channel, ebno_db, bit_errors, outcome, reliability, attempt	One row per frame (outcome: 0 OK, 1 detected, 2 undetected, 3 false reject)
runs	id, started_at, mode, notes	Live sessions, bulk populations, benchmarks
benchmark	profile, scheme, ebno_db, frames, fer, uer, goodput ...	Aggregated Monte-Carlo results
adaptive_log	ts, burst_index, chosen_profile, true_channel	Decisions of the adaptive controller
v_summary (view)	scheme, channel, ebno_db, counts	Per-scheme summary for queries

Table 3 - Database schema (data/guardcode.db)

4.2 Live dashboard

A background thread transmits frames continuously through the selected channel with all six receivers side by side. Each frame is written to the database, and a snapshot is pushed to the browser twice a second. The operator can change Eb/N0, switch between AWGN, burst and fading, trigger a 20-second burst storm, and change the frame rate. The page shows:

- a scoreboard with FER, BER, detected and undetected frames and goodput;
- live charts of cumulative undetected errors, rolling FER and goodput;
- the adaptive controller's burst index;
- a feed of undetected-error events.

The /explorer page runs read-only SQL with preset queries.

5. Simulation Setup

Parameter	Value
Payload / CRC / frame	32 bits / CRC-8 (x^8+x^2+x+1) / 40 bits
Channel symbols per frame	70 for all coded schemes (rate 32/70); 40 uncoded
Modulation reference	Real-valued unit-energy symbols (BPSK-equivalent for Hamming/uncoded)
Eb/N0 range	0 to 7 dB (energy per information bit)
Burst channel	Gilbert-Elliott: P(G->B) = 0.03, P(B->G) = 0.25, +8 dB noise in Bad state
Frames per operating point	3,00,000
Gate calibration	Both gates reject 1 % of correctly decoded frames at 3 dB (equal ARQ cost)
ARQ model	Selective repeat; goodput = correctly delivered info bits per channel

Parameter	Value
	use

Table 4 - Simulation parameters

6. Results

6.1 Error-rate performance

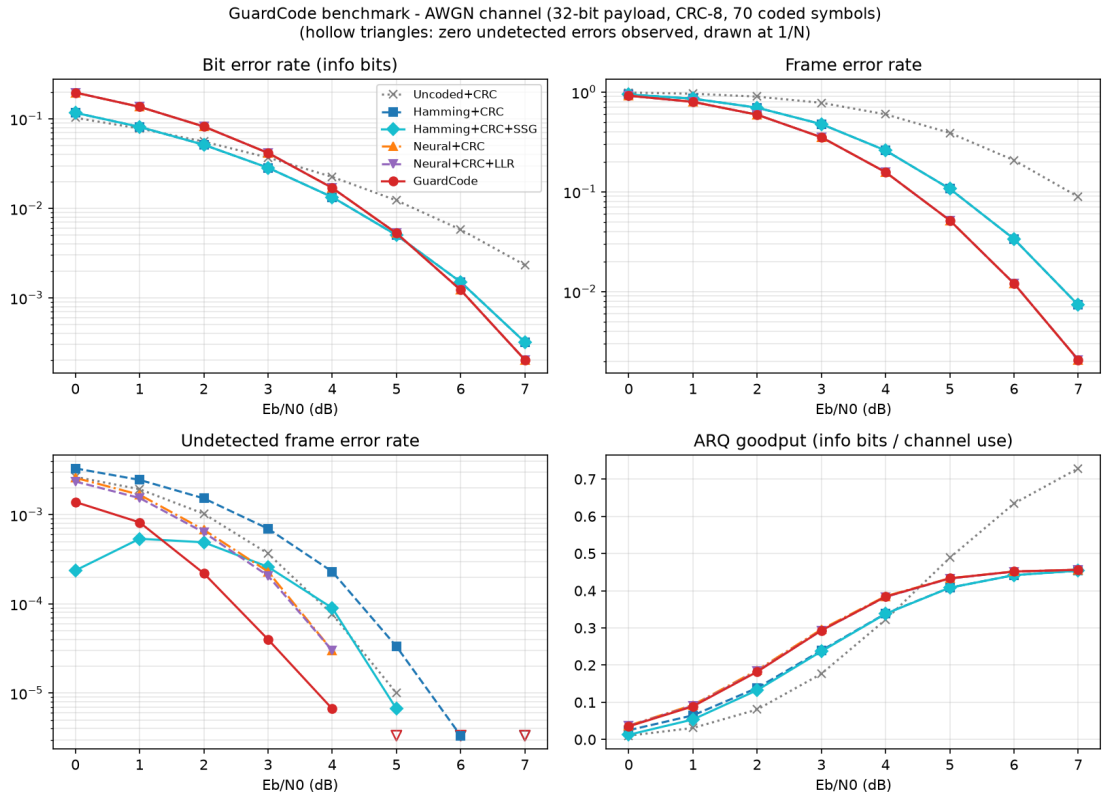


Figure 2 - AWGN channel: BER, FER, undetected FER and goodput

The learned code beats Hamming(7,4) at the same rate. At 4 dB its FER is 0.157 vs 0.260, and at 6 dB it is 0.0120 vs 0.0336, a gain of roughly 1 dB. This raises ARQ goodput. Its BER is higher at very low SNR because a wrong byte decision costs several bits; the frame-level metrics matter more for ARQ.

Scheme	0 dB	1 dB	2 dB	3 dB	4 dB	5 dB	6 dB	7 dB
Uncoded+CRC	0.9871	0.9613	0.8992	0.7793	0.5975	0.3895	0.2070	0.0891
Hamming+CRC	0.9455	0.8581	0.6968	0.4767	0.2596	0.1072	0.0336	0.0073
Hamming+CRC+SSG	0.9455	0.8581	0.6968	0.4767	0.2596	0.1072	0.0336	0.0073
Neural+CRC	0.9182	0.7969	0.5922	0.3523	0.1574	0.0517	0.0120	0.0021
Neural+CRC+LLR	0.9182	0.7969	0.5922	0.3523	0.1574	0.0517	0.0120	0.0021
GuardCode	0.9182	0.7969	0.5922	0.3523	0.1574	0.0517	0.0120	0.0021

Table 5a - Frame error rate, AWGN

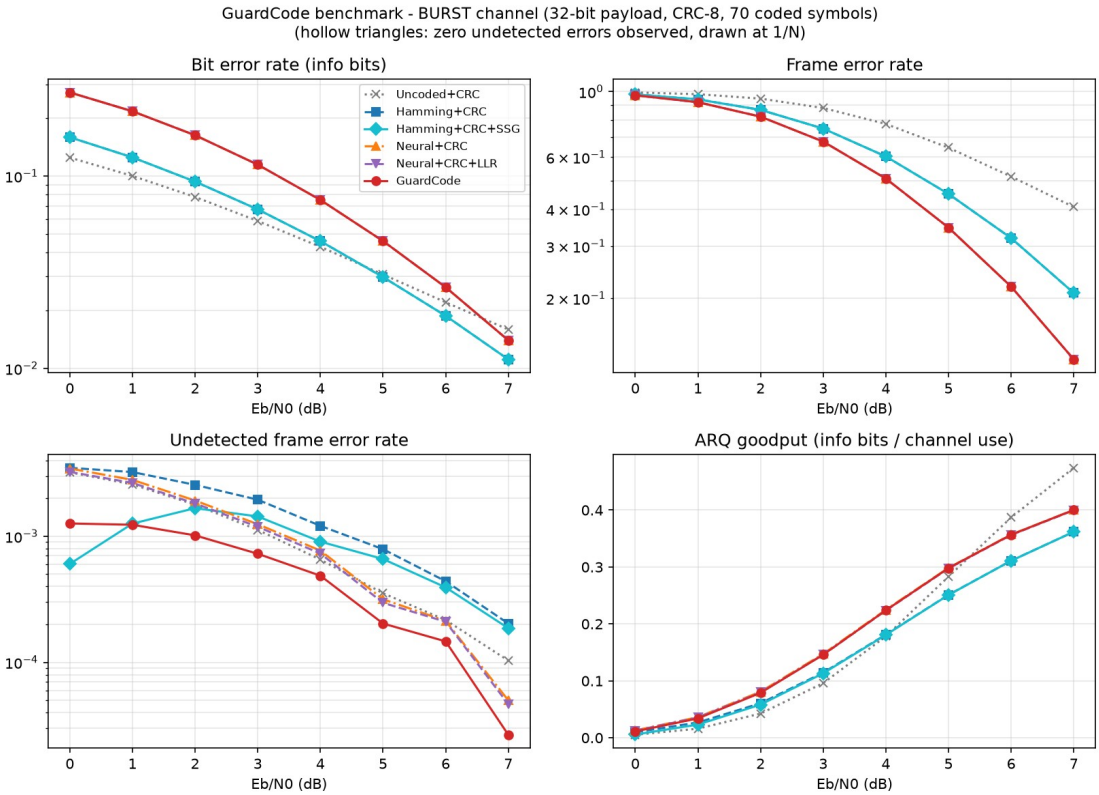


Figure 3 - Gilbert-Elliott burst channel

6.2 Undetected errors at equal retransmission cost

Scheme	Frames	Frame errors	Undetected	% of errors undetected	Ledger estimate (accepted)
Uncoded+CRC	24,00,000	14,73,042	1,817	0.123	-
Hamming+CRC	24,00,000	10,15,487	2,472	0.243	-
Hamming+CRC+SSG	24,00,000	10,15,487	486	0.048	483
Neural+CRC	24,00,000	8,64,837	1,559	0.180	-
Neural+CRC+LLR	24,00,000	8,64,837	1,428	0.165	-
GuardCode	24,00,000	8,64,837	742	0.086	132

Table 5b - Pooled over 0-7 dB, AWGN

Scheme	Frames	Frame errors	Undetected	% of errors undetected	Ledger estimate (accepted)
Uncoded+CRC	24,00,000	18,44,067	3,006	0.163	-
Hamming+CRC	24,00,000	15,35,751	4,161	0.271	-
Hamming+CRC+SSG	24,00,000	15,35,751	2,135	0.139	284
Neural+CRC	24,00,000	13,77,323	3,229	0.234	-
Neural+CRC+LLR	24,00,000	13,77,323	3,054	0.222	-
GuardCode	24,00,000	13,77,323	1,531	0.111	1621

Table 5c - Pooled over 0-7 dB, burst channel

On AWGN, GuardCode delivers 742 undetected frames against 1428 for the same decoder with the classic threshold test (48 % fewer) and 1559 with plain CRC acceptance (52 % fewer). On the burst channel the reductions are 50 % and 53 %. Applying the same guard to the Hamming receiver removes 80 % (AWGN) and 49 % (burst) of its undetected errors, which confirms that the method does not depend on the decoder. The price is small because the gates were calibrated to the same false-rejection budget. From 3 dB upward, where the link is usable, ARQ goodput stays within about 1 % of the unguarded receiver. At 0-2 dB the cost rises to a few percent, but goodput is already very low there.

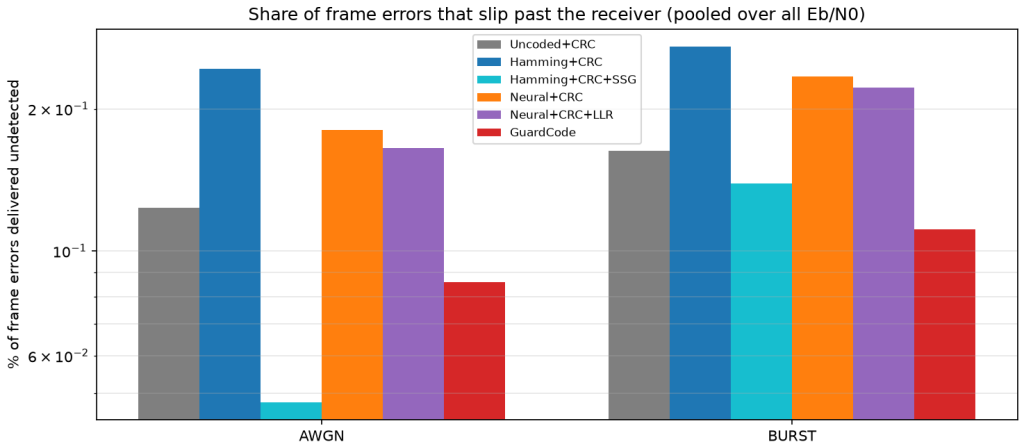


Figure 4 - Share of frame errors delivered undetected

6.3 Trade-off curves

Figures 5 and 6 sweep the gate threshold. The x-axis is the percentage of good frames sent back (extra retransmissions); the y-axis is the remaining undetected error rate. Each decoder has three curves:

- the classic min-|LLR| test (dashed);
- the bit-wise SSG variant (dotted);
- the group/WHT SSG used in GuardCode (solid).

The group form is consistently the lowest, which shows the gain comes from using the code's structure correctly, not from the threshold alone.

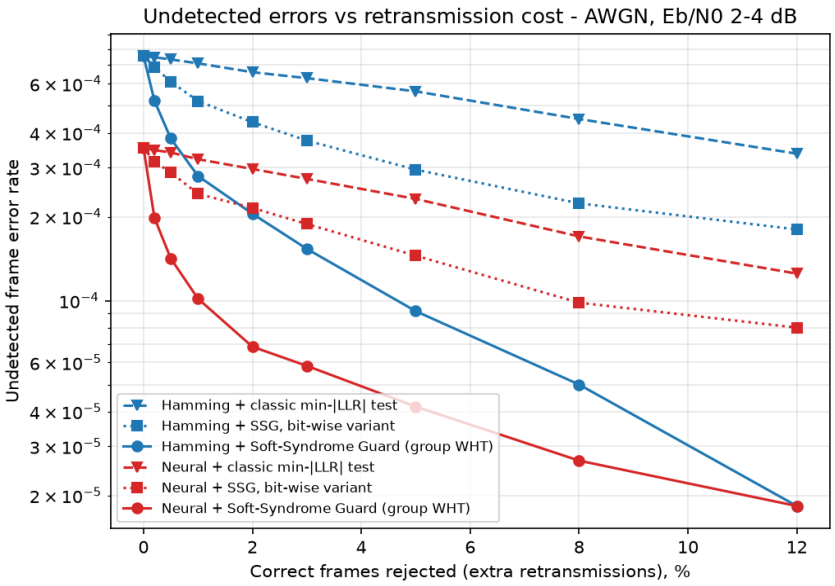


Figure 5 - Undetected error rate vs retransmission cost (AWGN, 2-4 dB)

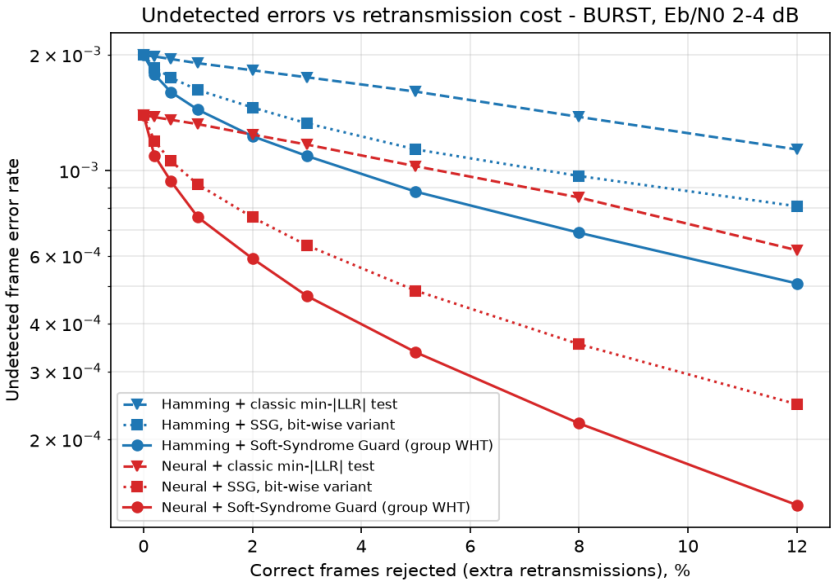


Figure 6 - Same, burst channel

Decoder (channel)	**No gate**	**LLR @1 %**	**SSG @1 %**	**LLR @5 %**	**SSG @5 %**
Neural (AWGN)	3.53e-4	3.22e-4	1.02e-4	2.32e-4	4.17e-5
Hamming (AWGN)	7.55e-4	7.10e-4	2.80e-4	5.63e-4	9.17e-5
Neural (BURST)	1.39e-3	1.32e-3	7.55e-4	1.03e-3	3.37e-4
Hamming (BURST)	1.99e-3	1.90e-3	1.44e-3	1.60e-3	8.80e-4

Table 6 - Undetected frame error rate at 1 % and 5 % false-reject budgets

6.4 Calibration and the risk ledger

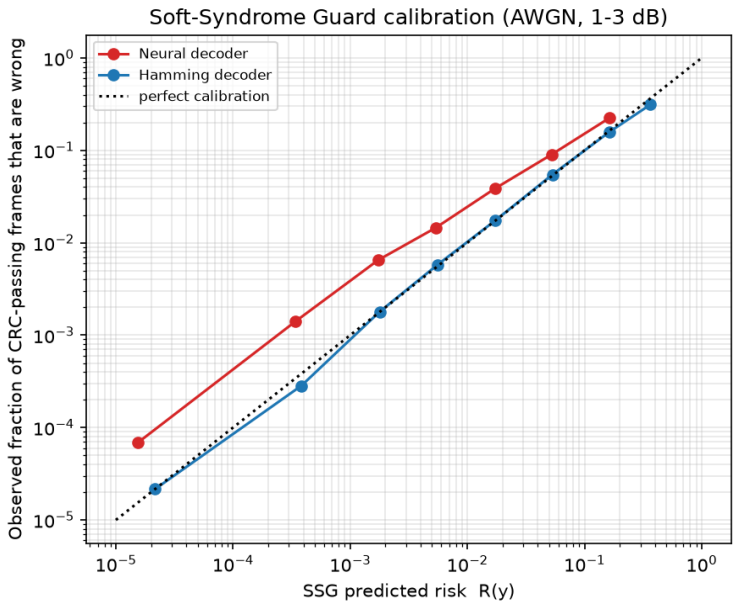


Figure 7 - Predicted risk vs observed error frequency

Frames are binned by predicted risk R(y), and each bin's observed fraction of wrong frames is plotted.

- **Hamming receiver:** its posteriors are exact on AWGN and its points lie on the diagonal. Summed over all CRC-passing frames between 1 and 3 dB, it predicted 934 undetected errors and 923 occurred, so R(y) is a true probability here.
- **Neural receiver:** its curve runs parallel to the diagonal but a factor of about 2-4 higher (248 predicted vs 534 observed). The network is somewhat over-confident, so the absolute risk is under-stated. The ordering of frames by risk is preserved, which is what the gate needs; this is why the guard still works well with it.

The ledger is only as good as the decoder's posteriors. Among frames the guard **accepted** (last column of Tables 5b and 5c), the Hamming receiver, whose posteriors are exact on AWGN, predicted 483 undetected errors against 486 observed. The neural decoder under-counts on AWGN (132 predicted vs 742 observed): the errors that survive the gate are confidently-wrong decisions, so its posteriors are over-confident in that tail. The Hamming receiver also under-counts on the burst channel, where its AWGN noise model is wrong. Better calibration of the neural decoder is listed as future work.

6.5 Channel-adaptive operation



Figure 8 - Profile switching when the channel changes AWGN -> burst -> AWGN (4 dB)

The burst index separates the channel classes. The controller switched 2 times in the three-phase scenario and had the matching profile active in 100 % of the blocks. Goodput during the burst phase was 0.223 adaptive vs 0.208 with the fixed AWGN profile; in the AWGN phases it was 0.384 vs 0.385. The gain from switching profiles is modest with the current burst-trained model; the main benefit is that each profile carries its own calibrated threshold.

6.6 Negative result: training-time guard loss

A training-time version of the same idea was also tried: a differentiable loss that penalises the probability of CRC-blind errors during training (guardcode/losses.py). Fine-tuning with weights $\lambda = 5$ and 50 did not change the measured share of undetected errors (0.106 %, 0.101 % and 0.099 % for $\lambda = 0$, 5 and 50 at 2-4 dB). With byte-level outputs, a single wrong byte is always caught by CRC-8, and the multi-byte coincidences that escape cannot be shaped by per-byte posteriors. This variant is therefore not claimed. The models are kept in models/ablation/ for reproducibility.

7. Patentability Assessment

Criterion	Assessment
Novelty	No document found that uses the dual-code expansion of a standard CRC to compute the posterior probability that a CRC-passing frame is wrong, and uses it as an ARQ acceptance gate with a risk ledger and per-profile thresholds.
Inventive step	Main risk. An examiner may combine reliability-output decoding (ROVA, Forney) with Gallager's parity-probability formula. Counter-arguments: neither applies the formula to an outer CRC to condition acceptance; there is a measured technical effect at equal retransmission cost; and the method is decoder-agnostic.
Industrial applicability	Strong. Drops into existing receivers (Wi-Fi, BLE, 5G control channels, industrial IoT) without changing the standard frame or polynomial.
Section 3(k), Indian	Claims are drafted as a receiver method and apparatus with a technical effect (fewer

Criterion	Assessment
Patents Act	corrupted frames delivered), not as an algorithm per se.

Table 7 - Patentability

A draft provisional specification with 12 claims is provided in docs/GuardCode_Provisional_Patent_Draft.docx. Recommended next steps:

- Have the VIT IPR cell or a registered patent agent run a professional search on InPASS and Espacenet, using CPC classes H03M13/09, H04L1/00 and H04L1/18.
- File a provisional application to secure the priority date.
- Complete the specification within 12 months.

A granted patent cannot be promised. Examination in India typically takes 2-4 years, and expedited examination is available to students and start-ups.

8. Conclusion and Future Work

GuardCode shows that the CRC on a link carries more information than a pass/fail bit. Combined with soft decoder output, the CRC's exact algebraic structure tells the receiver how likely a passing frame is to be wrong, and acting on that estimate removes a large share of undetected errors at no extra bandwidth. The work also produced:

- a neural channel code that beats Hamming(7,4) by about 1 dB;
- a live monitoring dashboard;
- a frame-level database with more than a million records;
- a draft patent.

Future work:

- extend to CRC-16/24/32 with a fast Walsh-Hadamard evaluation;
- apply SSG to LDPC/polar decoders and to HARQ soft combining;
- make the decoder SNR-aware so that posteriors stay calibrated on burst channels;
- implement the guard on an SDR testbed.

References

1. B. A. Forouzan, Data Communications and Networking, 5th ed., McGraw-Hill (course text: error detection/correction and ARQ).
2. C.-Y. Lou, B. Daneshrad, R. D. Wesel, "Convolutional-code-specific CRC code design," IEEE Trans. Communications, 2015 (arXiv:1506.02990).
3. "Autoencoders with list decoding for reliable data transmission over noisy channels," US Patent Application 2023/0198672 A1.
4. "Hierarchical deep channel coding," US Patent 12,155,482 B2.
5. Y. Jiang, H. Kim, H. Asnani, S. Kannan, S. Oh, P. Viswanath, "Turbo Autoencoder: Deep learning based channel codes for point-to-point communication channels," NeurIPS 2019.
6. A. R. Raghavan, C. W. Baum, "A reliability output Viterbi algorithm with applications to hybrid ARQ," IEEE Trans. Information Theory, 1998.

7. G. D. Forney, "Exponential error bounds for erasure, list, and decision feedback schemes," IEEE Trans. Information Theory, 1968.
8. R. G. Gallager, Low-Density Parity-Check Codes, MIT Press, 1963.
9. E. N. Gilbert, "Capacity of a burst-noise channel," Bell System Technical Journal, 1960; E. O. Elliott, "Estimates of error rates for codes on burst-noise channels," BSTJ, 1963.
10. "Error detection strategies for CRC-concatenated polar codes under successive cancellation list decoding," DLR, 2023.

Appendix A - How to Run

```
C:\Projects\Computer_Networks
  setup.bat            one-click: venv, dependencies, benchmark import, 1M-frame
database
  run_dashboard.bat    live dashboard at http://127.0.0.1:5000  (explorer: /explorer)
  run_experiments.bat  full benchmark (2M frames/point) + analyses
  retrain_models.bat   retrain neural profiles (PyTorch)

Manual:
  .venv\Scripts\python -m guardcode.evaluate --frames 300000
  .venv\Scripts\python scripts\analysis.py
  .venv\Scripts\python scripts\populate_db.py --frames 1000000
  .venv\Scripts\python dashboard\app.py
```

Appendix B - Core Code: Soft-Syndrome Guard

```
Using group posteriors (instead of independent bits) keeps the correlation between
bits of the same byte/block, which makes R(y) far better calibrated.

The residual risk  R(y) = 1 - P(b = x | y, Hb = 0)  is the posterior probability that
a
frame which PASSES the CRC is nevertheless wrong. The guard rejects (requests ARQ
retransmission of) CRC-passing frames with R(y) > tau. No extra bits are sent and the
CRC polynomial is not modified.
import numpy as np


_HAD = {}


def hadamard(n):
    """Sylvester Hadamard matrix with entries (-1)^popcount(s & v)."""
    if n not in _HAD:
        v = np.arange(n)
        bits = np.bitwise_and(v[:, None], v[None, :])
        par = np.zeros_like(bits)
        while bits.any():
            par ^= bits & 1
            bits >>= 1
        _HAD[n] = (1 - 2 * par).astype(np.float64)
    return _HAD[n]


def fwht(a):
    """Walsh-Hadamard transform along the last axis (length 2^m), unnormalised:
    out[s] = sum_v a[v] * (-1)^{popcount(s & v)}.  (A dense matrix product is the
    fastest form in numpy for the group sizes used here, 2^m <= 256.)"""
    a = np.asarray(a, dtype=np.float64)
    n = a.shape[-1]
    return (a.reshape(-1, n) @ hadamard(n)).reshape(a.shape)
```

```
class SoftSyndromeGuard:
    def __init__(self, crc):
        self.crc = crc
        self.W = crc.W.astype(np.int64)            # (2^r, n) rows u^T H
        self._idx = {}

    def _mask_index(self, m):
        """(U, G) index of s_c(u) for groups of m bits (MSB first inside a group)."""
        if m not in self._idx:
            U, n = self.W.shape
            G = n // m
            w = self.W.reshape(U, G, m)
            weights = 1 << np.arange(m - 1, -1, -1)
            self._idx[m] = (w * weights).sum(axis=2)    # (U, G)
        return self._idx[m]

    def risk_groups(self, P):
        """P: (B, G, 2^m) group posteriors. Returns (decision values (B,G), risk
(B,))."""
        P = np.asarray(P, dtype=np.float64)
        B, G, M = P.shape
        m = int(np.log2(M))
        P = P / P.sum(axis=2, keepdims=True)
        dec = P.argmax(axis=2)
        log_px = np.log(np.maximum(np.take_along_axis(P, dec[..., None], axis=2)[...,
0], 1e-300)).sum(axis=1)
        F = fwht(P)                                    # (B, G, M)
        idx = self._mask_index(m)                      # (U, G)
        U = idx.shape[0]
        if m == 1:
            # single-bit groups: prod_i t_i^{W_ui} with t = 1 - 2p, via log-magnitude
/ sign matmuls
            t = F[:, :, 1]
            Wf = idx.astype(np.float64).T                  # (G, U)
            mag = np.exp(np.log(np.maximum(np.abs(t), 1e-300)) @ Wf)
            sign = 1.0 - 2.0 * np.mod((t < 0).astype(np.float64) @ Wf, 2)
            pz = (sign * mag).mean(axis=1)
        else:
            acc = np.ones((B, U))
            for c in range(G):                         # product over groups, one
gather each
                acc *= F[:, c, :][:, idx[:, c]]
            pz = acc.mean(axis=1)
        pz = np.maximum(pz, np.exp(log_px))            # numerical guard: P(code) >=
P(x)
        risk = np.clip(1.0 - np.exp(log_px) / pz, 0.0, 1.0)
        return dec, risk

    # bit-level version (groups of 1 bit) kept for reference / ablation
    def risk(self, p1, bits=None):
        p1 = np.clip(np.asarray(p1, dtype=np.float64), 1e-15, 1 - 1e-15)
        P = np.stack([1 - p1, p1], axis=2)
        return self.risk_groups(P)[1]
```